# VALIDATION DES PRINCIPES D’ARCHITECTURE

Preuve de concept - orientation hospitalière en situation d’urgence

Ce document évalue l’alignement du PoC MedHead sur les principes métier, techniques et méthodologiques du Consortium. Chaque appréciation repose sur des éléments observables, dans les limites d’un démonstrateur isolé des systèmes de production.

| 14 | 03 | 02 |
|---|---|---|
| principes examinés | démontrés sur périmètre | non démontrés |

## PÉRIMÈTRE DES PREUVES

Backend Java 17 / Spring Boot 3 • Frontend Vue.js • Données H2 simulées • API REST/OpenAPI • CI GitHub Actions • Docker • Tests automatisés et JMeter

**Référentiel logiciel :** https://github.com/naegel23/P11-MedHead-PoC

**Référentiel architectural :** https://github.com/naegel23/P11-MedHead-Architecture

SYNTHÈSE / DÉCISION

# Synthèse des validations

Une appréciation de principe ne vaut pas certification réglementaire, qualification clinique ou garantie de disponibilité opérationnelle.

| 03 | 09 | 02 |
|---|---|---|
| **Démontrés** | **Partiels** | **Non démontrés** |
| Sur le périmètre réellement exercé. | Des preuves existent ; des exigences restent ouvertes. | Fonctionnalités ou contrôles non intégrés au PoC. |

## LECTURE DE LA DÉCISION

**Conclusion :** le PoC étaye la faisabilité du service d'orientation hospitalière et démontre la capacité à intégrer, tester et livrer du code reproductible. Il ne prouve pas la continuité clinique en production, la sécurité complète ni la chaîne événementielle de la plateforme cible.

### Points établis

API de recherche, filtres, base H2 de démonstration, interface Vue.js, tests automatisés, chaîne CI et exécution Docker. Charge locale vérifiée sur un endpoint sans routage externe.

### Points à consolider

Tolérance aux pannes, protection des accès, résilience du fournisseur de routage, automatisation E2E, pilotage des KPI métier et tests sur une plateforme représentative.

## CONVENTION DE LECTURE

| | |
|---|---|
| **Démontré** | Évidence vérifiable et représentative du périmètre annoncé. |
| **Partiellement démontré** | Un aspect du principe est couvert, mais le résultat n'est pas complet. |
| **Non démontré** | Aucune mise en œuvre suffisante pour conclure à la validation. |

TABLEAU DE CONFORMITÉ

# Matrice des 14 principes

Les statuts portent sur le PoC, et non sur la conformité de l'architecture cible entière.

| ID / PRINCIPE | | STATUT | JUSTIFICATION |
|---|---|---|---|
| A1 | Primauté des principes | Partiel | Référentiels partagés ; gouvernance d'adhésion non auditée |
| A2 | Avantage pour l'entreprise | Partiel | Valeur métier plausible, mais indicateurs non mesurés |
| A3 | Conformité lois et règlements | Partiel | Pas de patient réel ; conformité intégrale non auditée |
| A4 | Serment d'Hippocrate | Partiel | Démonstrateur isolé ; bénéfice clinique non mesuré |
| B1 | Continuité des activités critiques | Partiel | Charge locale validée ; pas de reprise ni de redondance |
| B2 | Séparation des préoccupations | Validé | Couches applicatives identifiables et testables |
| B3 | Intégration et livraison continues | Partiel | CI opérationnelle ; production non déployée |
| B4 | Tests automatisés appropriés | Partiel | Tests backend et frontend ; E2E automatisé incomplet |
| B5 | Sécurité shift-left | Non validé | Authentification, autorisation et secrets à consolider |
| B6 | Architecture événementielle | Non validé | Bus et publication d'événements non intégrés |
| C1 | Adaptation de l'ADM TOGAF | Partiel | Traçabilité documentaire ; gouvernance complète à formaliser |
| C2 | Référentiel centralisé | Validé | Dépôts code et architecture disponibles |
| C3 | Normes ouvertes | Partiel | REST et OpenAPI ; interopérabilité globale non vérifiée |
| C4 | Apprentissage par PoC | Validé | Hypothèses, campagnes de tests et limites explicitées |

**Priorité :** les principes B1 à B4 constituent le socle principal de validation de cette preuve de concept. B5 et B6 restent des conditions structurantes pour une industrialisation.

PRINCIPES MÉTIER / A1-A2

# Gouvernance et valeur partagée

Les principes métier évaluent la cohérence collective des décisions et leur contribution à la mission du Consortium.

## A1 Primauté des principes

Partiellement démontré

**Portée du principe.** Les règles d’architecture doivent s’appliquer de manière cohérente à toutes les entités du Consortium.

### CONSTATS ET PREUVES

- Le code et les livrables architecturaux sont accessibles dans des dépôts identifiés.
- Les exigences, tests et écarts sont documentés pour permettre une revue commune.

**Limite.** La publication des documents ne démontre pas, à elle seule, leur adoption formelle par tous les partenaires.

**Suite requise.** Faire valider les critères d’acceptation et les exceptions par les parties prenantes.

## A2 Maximiser les avantages pour l’entreprise

Partiellement démontré

**Portée du principe.** Les choix doivent renforcer les capacités communes et améliorer la prise en charge des patients.

### CONSTATS ET PREUVES

- Le service combine localisation, spécialité médicale et disponibilité simulée des lits.
- La solution expose des interfaces réutilisables et favorise l’intégration progressive.

**Limite.** Aucune campagne clinique ne vérifie un gain réel de délai ou de qualité de décision.

**Suite requise.** Piloter des scénarios métier représentatifs avec indicateurs de qualité des orientations et délais de traitement.

PRINCIPES MÉTIER / A3-A4

# Conformité et responsabilité clinique

L’absence de données patients réelles dans la PoC limite les risques, sans dispenser des contrôles nécessaires en exploitation.

## A3 Conformité aux lois et règlements

Partiellement démontré

**Portée du principe.** Le traitement des données et le fonctionnement du système doivent respecter les réglementations et politiques applicables.

### CONSTATS ET PREUVES

- Aucun dossier patient ni donnée clinique nominative n’est connecté au démonstrateur.
- Les disponibilités de lits et les associations de spécialités sont des valeurs simulées.

**Limite.** L’absence de données patients ne constitue pas une attestation globale de conformité RGPD, NHS ou cybersécurité.

**Suite requise.** Formaliser l’analyse des données, les obligations applicables, les contrôles d’accès et la conservation des traces.

## A4 Adhésion au serment d’Hippocrate

Partiellement démontré

**Portée du principe.** La sûreté des patients et la prévention des conséquences dommageables priment dans les décisions.

### CONSTATS ET PREUVES

- Le PoC fonctionne en environnement non productif, sans décision médicale réelle.
- Les limites de fiabilité des données et du routage sont explicitées.

**Limite.** La pertinence médicale des orientations et leurs effets sur la santé n’ont pas été évalués sur le terrain.

**Suite requise.** Organiser une validation clinique, des mécanismes de confirmation humaine et des procédures de repli.

PRINCIPES TECHNIQUES / B1

# Continuité des services critiques

Statut : partiellement démontré. Les mesures portent sur la capacité du backend à répondre, pas sur la continuité de soins en production.

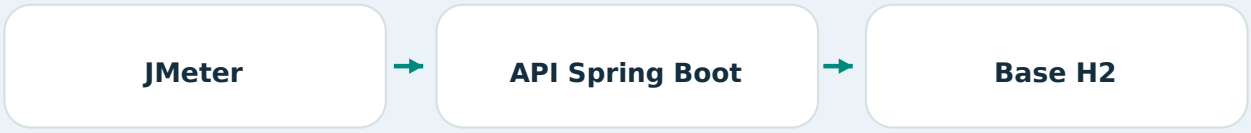
PARTIELLEMENT DÉMONTRÉ

## PREUVE EXPÉRIMENTALE DE PERFORMANCE

| 100 000 | 998,93 | 0 % | 137 ms |
|---|---|---|---|
| requêtes traitées | requêtes/seconde | erreurs mesurées | latence maximale |

**Conditions du scénario :** 100 utilisateurs virtuels, 1 000 itérations par utilisateur, objectif de charge de 800 requêtes/s. Point d'entrée testé : **/hospital/free/speciality/pharmacology/range/-2.0115245/52.00514984/100**, sans recours au fournisseur de routage externe. P95 : 15,95 ms ; P99 : 50 ms.

### PÉRIMÈTRE DU TEST




## RISQUES DE CONTINUITÉ ENCORE OUVERTS

- **Routage externe :** la dépendance OpenRouteService peut répondre HTTP 429, limitant les parcours de recommandation.
- **Résilience :** aucune validation de bascule automatique, redondance active ni restauration après sinistre.
- **Montée en charge :** les résultats locaux ne démontrent ni SLA ni disponibilité de la chaîne complète.

### Condition de validation complémentaire

Prévoir des tests de panne, de reprise, de saturation du service de routage et de déploiement multi-instance, avec des objectifs SLI/SLO définis.

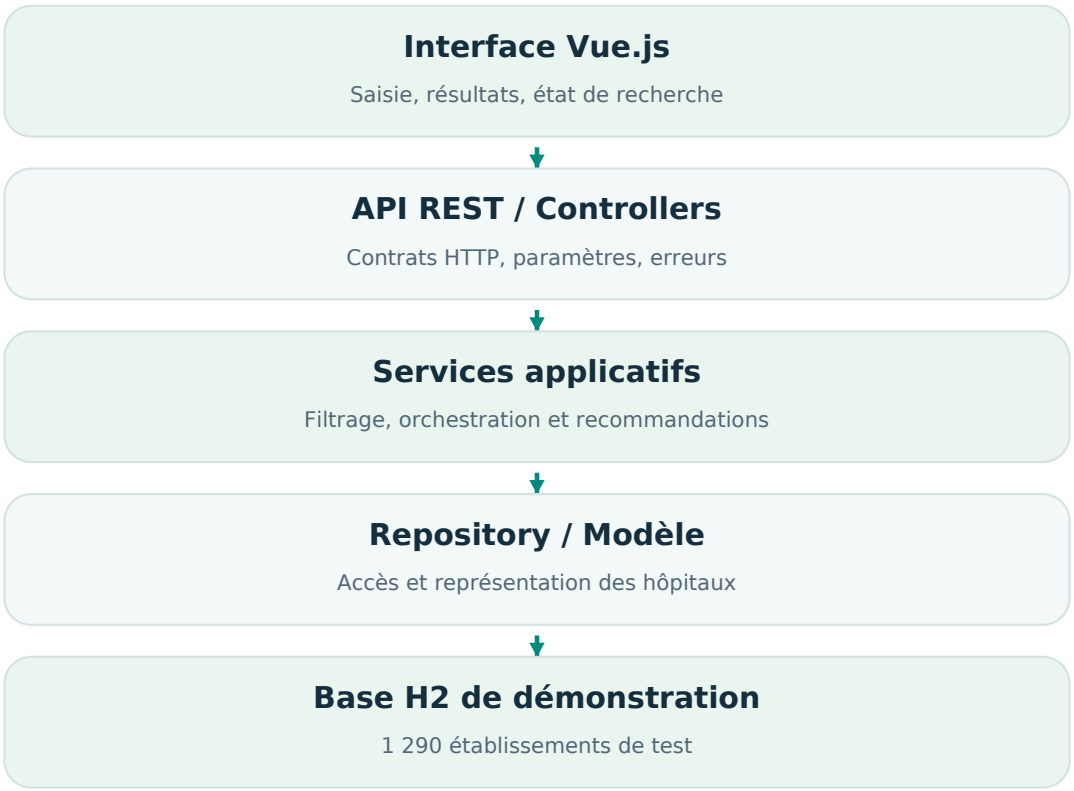
PRINCIPES TECHNIQUES / B2

# Séparation des préoccupations

Statut : démontré sur le périmètre de la PoC. Le découpage fonctionnel facilite la compréhension, les tests et l’évolution du service.

DÉMONTRÉ SUR LE PÉRIMÈTRE

## ARCHITECTURE LOGIQUE MISE EN ŒUVRE




**Frontière technique spécifique :** le calcul d’itinéraire interroge OpenRouteService depuis le backend. Il ne s’agit ni d’une base de lits temps réel ni d’un composant interne résilient.

### Portée de la validation

La séparation contrôleur / service / persistance est observable. Le découpage en plusieurs microservices indépendants et leur orchestration appartiennent à l’architecture cible.

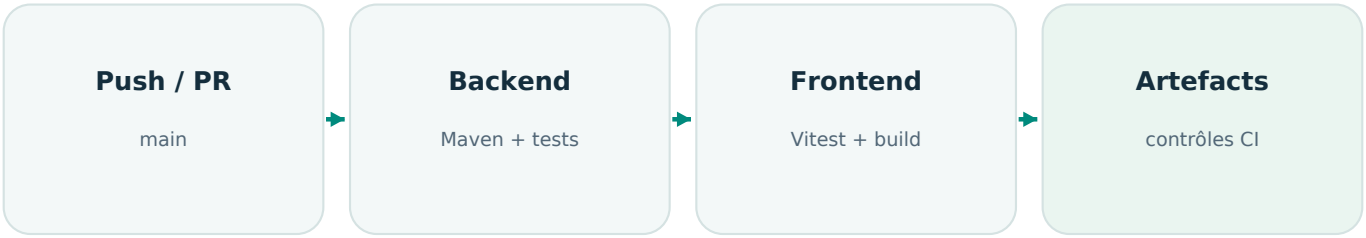
PRINCIPES TECHNIQUES / B3

# Intégration et livraison continues

Statut : partiellement démontré. La vérification continue est opérationnelle ; une livraison de production automatisée reste hors du périmètre validé.

PARTIELLEMENT DÉMONTRÉ

## CHAÎNE DE VALIDATION GITHUB ACTIONS




## ÉLÉMENTS CONSTATÉS

- Les changements et pull requests vers **main** déclenchent les contrôles du projet.
- Le backend est construit et vérifié sous **Java 17** ; sept tests backend sont validés.
- Le frontend Vue.js dispose de tests automatisés et d'une étape de build.
- Les journaux d'exécution GitHub Actions assurent une traçabilité des résultats.
- Le backend peut être construit et exécuté sous Docker avec un runtime Java 17.

### Écart à l'objectif de livraison continue

L'automatisation des builds et des tests ne prouve pas la présence d'un déploiement en environnement d'exploitation. Les approbations, la promotion d'artefacts et le rollback restent à définir.

**Prochaine étape :** séparer les jobs de vérification, publication d'image, déploiement en préproduction et contrôle post-déploiement ; conditionner toute promotion à des critères explicites.

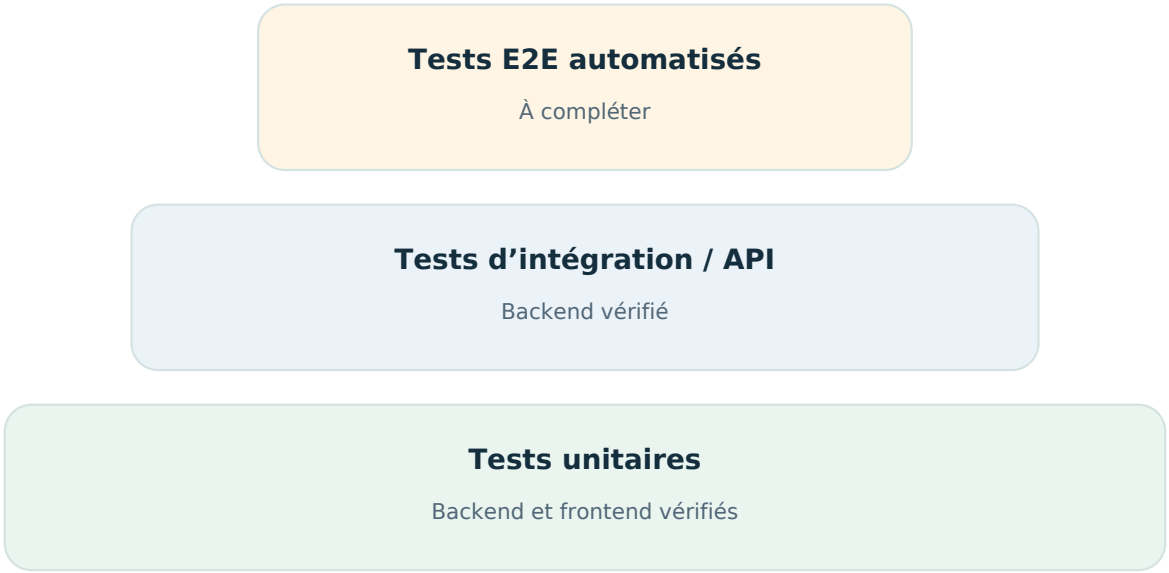
PRINCIPES TECHNIQUES / B4

# Tests automatisés et appropriés

Statut : partiellement démontré. Les campagnes couvrent les composants clés du PoC ; plusieurs exigences de bout en bout restent à automatiser.

PARTIELLEMENT DÉMONTRÉ

## PYRAMIDE DE VALIDATION ADAPTÉE AU POC




## TRACES DE VALIDATION

- **Backend :** 7 tests Maven réussis sur les scénarios vérifiés.
- **Frontend :** tests automatisés Vitest / Vue Test Utils ; parcours navigateur manuel avec recherche et affichage des réponses API.
- **Contrats :** spécification OpenAPI 3 accessible via /v3/api-docs et Swagger UI via /swagger-ui/index.html.
- **Performance :** campagne JMeter de 100 000 requêtes sans erreur sur un endpoint interne.

### Couverture restant à démontrer

Automatiser la chaîne interface → API → routage, les cas d’absence de lit, les réponses HTTP 429, les timeouts, les erreurs réseau et les scénarios de dégradation contrôlée.

PRINCIPES TECHNIQUES / B5-B6

# Sécurité et événements métier

Ces deux principes sont définis pour la plateforme MedHead, sans mise en œuvre suffisante dans la preuve de concept.

## B5 Sécurité de type « shift-left »

Non démontré

**Portée du principe.** Intégrer les exigences de sécurité dès les premiers incréments.

### CONSTATS ET PREUVES

- La PoC n’utilise pas de dossier patient réel ni de flux clinique nominatif.
- La surface HTTP et la dépendance à une API externe sont identifiées.

**Limite.** Les contrôles de production (authentification, autorisation, TLS, coffre de secrets et tests de sécurité systématiques) ne sont pas démontrés.

**Suite requise.** Gérer la clé du fournisseur de routage comme un secret ; introduire des contrôles de sécurité dans la CI et protéger les routes.

## B6 Extension par événements métier

Non démontré

**Portée du principe.** Les capacités métier doivent pouvoir publier et consommer des événements de manière découplée.

### CONSTATS ET PREUVES

- Le service expose actuellement ses fonctions au travers d’API REST.
- Un bus d’événements figure parmi les composants de l’architecture cible.

**Limite.** Aucune publication d’événements, file durable, consumer ni garantie de livraison n’est validée par le PoC.

**Suite requise.** Définir un contrat d’événement de réservation, puis vérifier publication, idempotence, reprise et supervision.

PRINCIPES MÉTHODOLOGIQUES / C1-C2

# Cadre TOGAF et référentiel

La documentation rend les choix vérifiables et partageables ; elle ne remplace pas la gouvernance de décision du Consortium.

## C1 Adaptation de l'ADM TOGAF

Partiellement démontré

**Portée du principe.** Utiliser un processus d'architecture adapté au risque et au périmètre d'une preuve de concept.

### CONSTATS ET PREUVES

- Exigences, stratégie de test, SBB, résultats et principes sont organisés par finalité.
- La démarche associe hypothèses, validation technique et analyse des écarts.

**Limite.** Les revues formelles, décisions d'arbitrage et approbations du cycle de gouvernance ne sont pas attestées.

**Suite requise.** Formaliser les décisions architecturales et les critères de passage vers l'étape suivante.

## C2 Référentiel architectural centralisé

Démontré

**Portée du principe.** Mettre à disposition des artefacts cohérents et traçables pour faciliter la collaboration.

### CONSTATS ET PREUVES

- Un dépôt de code dédié rassemble implémentation, tests et workflow GitHub Actions.
- Un dépôt architectural distinct expose les exigences, blocs de solution et résultats.

**Limite.** L'existence des dépôts valide l'accessibilité et la traçabilité documentaire dans le périmètre du PoC, pas une gouvernance organisationnelle complète.

**Suite requise.** Conserver un index documentaire, une convention de revue et des liens entre exigence, test et résultat.

PRINCIPES MÉTHODOLOGIQUES / C3-C4

# Normes ouvertes et apprentissage

Les solutions techniques sont examinées au regard de la réutilisabilité, de la transparence et des apprentissages obtenus.

## C3 Normes ouvertes et bonnes pratiques

Partiellement démontré

**Portée du principe.** Favoriser des contrats interopérables, des composants observables et une architecture modulaire.

### CONSTATS ET PREUVES

- L'API REST dispose d'une spécification OpenAPI 3 documentant ses routes.
- Le backend est empaquetable avec Docker et s'appuie sur un socle Java standard.

**Limite.** Les exigences étendues de la cible (service mesh, bus, IAM, observabilité transverse) ne sont pas démontrées.

**Suite requise.** Valider les contrats d'intégration et les normes de sécurité et d'observabilité retenues pour l'exploitation.

## C4 Apprendre grâce à une preuve de concept

Démontré

**Portée du principe.** Tester tôt les hypothèses risquées, avec des preuves mesurables et un environnement isolé.

### CONSTATS ET PREUVES

- Le PoC utilise un jeu de données hospitalières de démonstration, sans données réelles de patients.
- Les tests fonctionnels et de charge identifient les capacités démontrées et les risques résiduels.

**Limite.** Le PoC réduit l'incertitude technique ; il ne supprime pas les risques cliniques et d'intégration de la plateforme cible.

**Suite requise.** Poursuivre par des scénarios représentatifs et décider à partir d'indicateurs et de critères d'acceptation partagés.

HYPOTHÈSES ET INDICATEURS

# Validation des critères de succès

Les objectifs ci-dessous sont évalués séparément afin de ne pas déduire un bénéfice clinique de la seule performance informatique.

### Orientation appropriée

Non mesuré

**Cible :** Plus de 90 % des urgences
**Constat :** Non mesuré

Nécessite un échantillon médical qualifié et la comparaison aux décisions expertes.

### Délai de traitement

Non mesuré

**Cible :** 18,25 min → 12,00 min
**Constat :** Non mesuré

Requiert une étude du parcours de soins et une mesure des étapes opérationnelles.

### Performance du backend

Démontré

**Cible :** Moins de 200 ms à 800 req/s
**Constat :** Démontré localement

100 000 requêtes, 998,93 req/s, 0 % d'erreurs et latence max. de 137 ms sur l'endpoint filtré.

### Transparence des choix

Démontré

**Cible :** Normes et limites documentées
**Constat :** Démontré sur périmètre

Contrats d'API, tests, composants, risques et principes explicités dans les livrables.

### Mise en production

À compléter

**Cible :** Instructions et contrôles nécessaires
**Constat :** À compléter

Exécution Docker possible ; publication, secrets, reprise et procédures opérationnelles non qualifiées.

**Interprétation clé :** la condition de latence est démontrée sur une route de filtrage interne et dans les conditions de test déclarées. La chaîne de recommandation avec calcul d'itinéraire OpenRouteService n'est pas comprise dans cette validation de charge.

RISQUES ET RECOMMANDATIONS

# Plan de consolidation priorisé

Les actions sont ordonnées selon leur impact potentiel sur les patients, la fiabilité de la recommandation et la reproductibilité de la solution.

| | | |
|---|---|---|
| P1 | **Continuité du routage externe** | B1 |
| | Tester erreurs 429, timeouts, backoff et mode dégradé ; définir quotas et supervision. | |
| P1 | **Sécurité des accès et des secrets** | B5 / A3 |
| | Protéger secrets et clés, définir une authentification et vérifier TLS et autorisations. | |
| P1 | **Validation clinique du résultat** | A2 / A4 |
| | Évaluer pertinence des recommandations et délais sur un jeu clinique qualifié. | |
| P2 | **Reprise et haute disponibilité** | B1 |
| | Déployer des instances redondantes ; tester panne, reprise, sauvegarde et observabilité. | |
| P2 | **Tests de bout en bout** | B4 |
| | Automatiser les parcours UI-API-routage et les réponses d’erreur contrôlées. | |
| P2 | **Livraison reproductible** | B3 |
| | Ajouter publication d’image, préproduction, validations et procédure de rollback. | |
| P3 | **Événements et intégration** | B6 / C3 |
| | Définir les événements de réservation et tester leur consommation idempotente. | |
| P3 | **Gouvernance et traçabilité** | A1 / C1 |
| | Formaliser décisions, responsables, exceptions et conditions de passage à l’échelle. | |

### Condition de décision

Avant exploitation : valider les scénarios métier, la sécurité, la résilience et les critères d’acceptation.

RÉFÉRENCES ET CONCLUSION

# Décision de validation

Les preuves ci-dessous rendent les constats vérifiables, sans élargir artificiellement le périmètre réellement observé.

## EMPLACEMENTS DE VÉRIFICATION

### Code du PoC
github.com/naegel23/P11-MedHead-PoC

Sources backend/frontend, tests, Docker et workflows GitHub Actions.

### Référentiel d'architecture
github.com/naegel23/P11-MedHead-Architecture

Exigences, catalogue SBB, stratégie de test, validation et reporting.

### Documentation d'API
/v3/api-docs · /swagger-ui/index.html

Spécification et présentation des contrats de service localement.

### Résultats CI
GitHub Actions · branche main

Journaux des builds et des tests exécutés par le pipeline.

### Résultats de performance
Reporting des résultats du PoC MedHead

Mesures JMeter, conditions de charge et limites du test.

## CONCLUSION ARCHITECTURALE

Le PoC MedHead fournit une **validation technique favorable et circonscrite** : recherche d'établissements, découpage en couches, documentation API, tests automatisés, intégration continue et performance mesurée d'un endpoint backend.

Il ne permet pas de conclure à la conformité globale de la plateforme pour des interventions réelles. Les travaux prioritaires portent sur la **résilience**, la **sécurité**, la **validation clinique**, les **parcours E2E** et les **événements métier**.

**Décision proposée :** poursuivre vers une étape d'intégration et de qualification représentative, sous réserve d'un plan de traitement des risques et de critères d'acceptation approuvés.